

Machine-Verification of the HyMeKo Propositions

Symbolic proofs and property-tests against the real compiler (T-SMC appendix)

Aiko (Claude Code), for Dr. Csaba Hajdu

2026-06-28 (JST)

Abstract

The article’s four propositions are presented as proof-sketches-by-inspection, with the load-bearing one (Prop. 1) exiled to supplementary material — the #1 weakness a prior review flagged. We attack it mechanically. The propositions split by mathematical character, so the achievable rigor differs per proposition: the *algebraic* claims (storage bound, collision bound) admit full **symbolic proofs** (*sympy*); the *structural* claims (alias invariance, emitter purity/independence) are **property-tested against the real compiler** (the engine’s `parse_dsl`→`canonical.hash` / emitters), which tests the actual artifact rather than an idealisation. Result: **three of four propositions are machine-confirmed**, and the fourth surfaces a **precise, fixable correction** — Prop. 1’s *byte-equal emit* claim fails under sibling reordering (the emitters carry source-order IDs), although its content-address claim holds on every one of 800 tested instances. Tooling installed and pinned via `uv`: *sympy* 1.14, *z3* 4.16, *hypothesis* 6.15, *networkx* 3.6, *pygraphviz* 2.0.

1 Results at a glance

Prop.	Claim	Method	Outcome
P1 alias invariance	denotation-equal $\Rightarrow$ same IR <i>and</i> byte-equal emit	Hypothesis vs. real compiler, 800 reorderings $\times$ 4 fixtures	hash: verified (800/800) emit: corrected (0/400)
P2 content-address	h pure + collision-resistant	<i>sympy</i> bound + P1 mechanism	verified (+ stated assumption)
P3 proj.–emit indep.	emitters pure; k emits, 1 compile	determinism property-test	verified (4/4)
P4 storage overhead	$\rho - 1 = O(\log n/\bar{d}) \rightarrow 1$	<i>sympy</i> symbolic proof	verified (full proof)

2 P4 — storage overhead (full symbolic proof)

From the size equations $|H|_{\text{IR}} = (n + m)c_{\text{rec}} + m\bar{d}c_{\text{inc}}$ and $|H|_{\text{adj}} = m\bar{d}c_{\text{inc}}$, *sympy* derives $\rho = 1 + c \frac{n+m}{m\bar{d}}$ ($c = c_{\text{rec}}/c_{\text{inc}}$), substitutes the assumption $n \leq Km \log n$ to obtain $\rho - 1 \leq \frac{c(K \log n + 1)}{\bar{d}} = O(\log n/\bar{d})$, confirms $\lim_{\bar{d} \rightarrow \infty} (\rho - 1) = 0$, confirms strict monotone decrease ($\partial_{\bar{d}}(\rho - 1) < 0$), and reproduces the empirical witness exactly: with $n_{\text{pool}} = m = 200$, $c = 1$, $\rho - 1 = 2/\bar{d}$, giving $\rho = 2.00$ at $\bar{d} = 2$ and $\rho = 1.01$ at $\bar{d} = 200$. Every step is machine-checked (`verification/propositions/p4.storage.overhead.py`). This is a proof, not a sketch.

3 P2 — content-addressability (symbolic bound + a stated assumption)

P2 has two parts. (a) *h depends only on the canonical IR* — the same mechanism as P1, property-tested below at 800/800. (b) *distinct IRs collide negligibly* — a birthday bound on the 256-bit digest, proved symbolically: $P(\text{collision among } N) \leq N^2/2^{257}$, so $P \leq 2^{-128}$ while $N \leq 2^{64.5}$ (work factor 2^{128}); numerically $N=2^{32} \rightarrow 2^{-193}$, $N=2^{64} \rightarrow 2^{-129}$. *Honest caveats*: the random-oracle premise for Blake3 is a cryptographic *assumption*, not proved; and a 256-bit digest cannot be injective (pigeonhole), so equality-by-hash is *practical*, not absolute — which the article already concedes.

4 P1 & P3 — against the real compiler

We compile self-contained fixtures, apply denotation-preserving *sibling reorderings* (the strongest automatable rewrite), and compare against the engine. Distinct fixtures hash distinctly (fano b88715ed, generic 6a0c09d8, k4 0a8a903e, sunflower 194c71e4), so the test is non-degenerate.

Verified. The **canonical hash is invariant on all 800 reorderings** (4 fixtures $\times$ 200) — zero violations: Prop. 1’s content-address claim holds against the real compiler. Emitters are **deterministic** (4/4): re-compiling the same source in a fresh engine emits byte-identical DOT, so each emitter is a pure function of its input (Prop. 3’s independence then follows — a pure emitter that ignores the others can be run k times off one compile).

Corrected. Prop. 1’s *stronger* claim — denotation-equal sources emit *byte-identical* text — **fails under sibling reordering** (0/400 byte-equal DOT). The cause is precise: the emitters assign internal identifiers (n2, e5, ...) in *source-declaration order*, so a reordered source emits a DOT whose IDs are permuted (not merely the lines — the ID $\leftrightarrow$ label mapping changes). The graph is isomorphic; the serialisation is not byte-equal. *Only the hash is order-canonicalised; the stored IR and its emit retain source order.* The article’s existing `prop1_alias` test uses *alias-renaming* fixture pairs (where names canonicalise and byte-equality plausibly holds); it does not exercise sibling reordering, which is why this gap was invisible.

Remedy (for the article). Three options, in increasing strength: (i) *scope* the byte-equal claim to alias/import rewrites and state the sibling-reorder guarantee at the canonical-hash level; (ii) *canonicalise emit IDs* (assign by the canonical sort key before emitting), making byte-equality hold for reordering too; (iii) *weaken* the claim to “output equal up to isomorphism / equal canonical hash,” which holds universally. Option (i) is a one-paragraph edit; option (ii) is the principled fix.

5 The rigor ladder (kept honest)

- **Proved (symbolic):** P4 (storage), P2 collision bound — sympy, fully rigorous.
- **Property-verified (real artifact, falsification over 800 instances):** P1 canonical-hash invariance, P3 emitter determinism.
- **Assumed (not provable here):** Blake3 random-oracle (P2); sibling reordering taken as denotation-preserving *by* the engine’s own canonical hash.

- **Corrected:** P1 byte-equal emit under sibling reordering.
- **z3:** installed, but the four propositions have no natural decidable-SMT formulation that adds over `sympy`+property-testing; its value here is future work (signed-balance gauge properties, bounded model-checking the resolver), not these four.

6 What this buys the article

The appendix moves from “proof sketches by inspection” to “symbolically proved where algebraic, and property-verified over 800 instances *against the real compiler* where structural” — and the exercise *found a precise, fixable defect* the prose proof had hidden. This directly answers the review’s #1 finding, and suggests a genuine new angle: a **machine-checked canonical-IR appendix** (or a short companion artifact) is a stronger contribution than one more robot fixture. Scripts: `verification/propositions/{p1_alias_invariance, p2_content_birthday, p3_emit_purity, p4_storage_overhead}.py`.